

Notes on Chapter 22 - Arrow

The Caveman Coder

2025-08-13

Main idea: Using files in parquet format as an alternative to CSV.

Prerequisites

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.2      v tibble    3.3.0
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.1.0
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x dplyr::filter() masks stats::filter()
```

```
## x dplyr::lag()     masks stats::lag()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(arrow)
```

```
## Some features are not enabled in this build of Arrow. Run `arrow_info()` for more information.
```

```
##
```

```
## Attaching package: 'arrow'
```

```
##
```

```
## The following object is masked from 'package:lubridate':
```

```
##
```

```
##     duration
```

```
##
```

```
## The following object is masked from 'package:utils':
```

```
##
```

```
##     timestamp
```

```
library(dbplyr, warn.conflicts = FALSE)
```

```
library(duckdb)
```

```
## Loading required package: DBI
```

Getting the data

The data is 9GB large so it is not practical for me to download it. The code below can be used to download it:

```
dir.create("data", showWarnings = FALSE)
```

```
curl::multi_download(
```

```
"https://r4ds.s3.us-west-2.amazonaws.com/seattle-library-checkouts.csv",
"data/seattle-library-checkouts.csv",
resume = TRUE
)
```

Opening a dataset

Since the data is 9GB, it is not practical to load the whole thing into memory. Instead of using `read_csv()`, it is better to use `arrow::open_dataset()`:

```
seattle_csv <- open_dataset(
  sources = "data/seattle-library-checkouts.csv",
  col_types = schema(ISBN = string()),
  format = "csv"
)
```

When run, `open_dataset()` will scan a few thousand rows to figure out the structure of the dataset. Once the data has been scanned, it records what it's found and stops – it will only read further if tell it to (using commands, of course!). This is what we will see if we print the data:

```
seattle_csv
#> FileSystemDataset with 1 csv file
#> UsageClass: string
#> CheckoutType: string
#> MaterialType: string
#> CheckoutYear: int64
#> CheckoutMonth: int64
#> Checkouts: int64
#> Title: string
#> ISBN: string
#> Creator: string
#> Subjects: string
#> Publisher: string
#> PublicationYear: string
```

To reveal the structure, the function `glimpse()` can be used here:

```
seattle_csv |> glimpse()
#> FileSystemDataset with 1 csv file
#> 41,389,465 rows x 12 columns
#> $ UsageClass      <string> "Physical", "Physical", "Digital", "Physical", "Ph...
#> $ CheckoutType    <string> "Horizon", "Horizon", "OverDrive", "Horizon", "Hor...
#> $ MaterialType     <string> "BOOK", "BOOK", "EBOOK", "BOOK", "SOUNDDISC", "BOO...
#> $ CheckoutYear     <int64> 2016, 2016, 2016, 2016, 2016, 2016, 2016, 2016, 20...
#> $ CheckoutMonth    <int64> 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6,...
#> $ Checkouts        <int64> 1, 1, 1, 1, 1, 1, 1, 1, 4, 1, 1, 2, 3, 2, 1, 3, 2,...
#> $ Title            <string> "Super rich : a guide to having it all / Russell S...
#> $ ISBN              <string> "", "", "", "", "", "", "", "", "", "", "", "", "", ""...
#> $ Creator           <string> "Simmons, Russell", "Barclay, James, 1965-", "Tim ...
#> $ Subjects          <string> "Self realization, Conduct of life, Attitude Psych...
#> $ Publisher         <string> "Gotham Books,", "Pyr,", "Random House, Inc.", "Di...
#> $ PublicationYear   <string> "c2011.", "2010.", "2015", "2005.", "c2004.", "c20..."
```

Example using dplyr verbs on the data:

```
seattle_csv |>
  group_by(CheckoutYear) |>
  summarize(Checkouts = sum(Checkouts)) |>
  arrange(CheckoutYear) |>
  collect()
#> # A tibble: 18 × 2
#>   CheckoutYear Checkouts
#>   <int>         <int>
#> 1      2005     3798685
#> 2      2006     6599318
#> 3      2007     7126627
#> 4      2008     8438486
#> 5      2009     9135167
#> 6      2010     8608966
#> # i 12 more rows
```

The code above will work regardless of how large the underlying dataset is. But the execution may run “slow” due to the size of the dataset. This is where using the parquet format becomes a good idea.

The parquet format

Advantages of parquet

- Parquet files are usually smaller than the equivalent CSV file (due to its custom binary format that supports file compression).
- Parquet files store data in a way that records the type along with the data.
- Parquet files are column-oriented (organized column-by-column).
- Parquet files are “chunked”, which makes it possible to work on different parts of the file at the same time.

Disadvantages of parquet

- Parquet files are no longer “human readable” since it is in binary format. Reading it in raw form will yield “gibberish”.

Partitioning

There are no hard and fast rules about how to partition the dataset. But rule of thumb dictates the following:

- avoid files smaller than 20MB and larger than 2GB.
- avoid having more than 10,000 files.

Rewriting the Seattle library data (in parquet)

Steps:

1. define the partition using `dplyr::group_by()`
2. save the partitions to a directory with `arrow::write_dataset()`

```
pq_path <- "data/seattle-library-checkouts"

seattle_csv |>
```

```
group_by(CheckoutYear) |>
write_dataset(path = pq_path, format = "parquet")
```

Writing the seattle dataset to parquet takes about a minute but this initial “investment” pays off by making future operations much faster.

To see what was written to file:

```
tibble(
  files = list.files(pq_path, recursive = TRUE),
  size_MB = file.size(file.path(pq_path, files)) / 1024^2
)
#> # A tibble: 18 × 2
#>   files                                size_MB
#>   <chr>                                <dbl>
#> 1 CheckoutYear=2005/part-0.parquet      109.
#> 2 CheckoutYear=2006/part-0.parquet      164.
#> 3 CheckoutYear=2007/part-0.parquet      178.
#> 4 CheckoutYear=2008/part-0.parquet      195.
#> 5 CheckoutYear=2009/part-0.parquet      214.
#> 6 CheckoutYear=2010/part-0.parquet      222.
#> # i 12 more rows
```

The original 9GB single file has been rewritten into 18 parquet files and each file is between 100 MB and 300 MB; the total size is now around 4 GB.

Using dplyr with arrow

Using `open_dataset()` on the parquet files:

```
seattle_pq <- open_dataset(pq_path)
```

Applying query operations on `seattle_pq`:

```
query <- seattle_pq |>
  filter(CheckoutYear >= 2018, MaterialType == "BOOK") |>
  group_by(CheckoutYear, CheckoutMonth) |>
  summarize(TotalCheckouts = sum(Checkouts)) |>
  arrange(CheckoutYear, CheckoutMonth)
```

Printing out the query object yields:

```
query
#> FileSystemDataset (query)
#> CheckoutYear: int32
#> CheckoutMonth: int64
#> TotalCheckouts: int64
#>
#> * Grouped by CheckoutYear
#> * Sorted by CheckoutYear [asc], CheckoutMonth [asc]
#> See $.data for the source Arrow object
```

To see the results, we need to apply `collect()`:

```
query |> collect()
#> # A tibble: 58 × 3
#> # Groups:   CheckoutYear [5]
#>   CheckoutYear CheckoutMonth TotalCheckouts
```

```
#>           <int>           <int>           <int>
#> 1         2018             1         355101
#> 2         2018             2         309813
#> 3         2018             3         344487
#> 4         2018             4         330988
#> 5         2018             5         318049
#> 6         2018             6         341825
#> # i 52 more rows
```

Performance

Measuring how long it takes to perform a calculation or an operation on the original 9GB CSV dataset (using `system.time()`):

```
seattle_csv |>
  filter(CheckoutYear == 2021, MaterialType == "BOOK") |>
  group_by(CheckoutMonth) |>
  summarize(TotalCheckouts = sum(Checkouts)) |>
  arrange(desc(CheckoutMonth)) |>
  collect() |>
  system.time()
#>      user system elapsed
#> 11.951  1.297 11.387
```

Measuring how long it takes to perform the same calculation on the the data partitioned into 18 smaller parquet files:

```
seattle_pq |>
  filter(CheckoutYear == 2021, MaterialType = "BOOK") |>
  group_by(CheckoutMonth) |>
  summarize(TotalCheckouts = sum(Checkouts)) |>
  arrange(desc(CheckoutMonth)) |>
  collect() |>
  system.time()
#>      user system elapsed
#>  0.263  0.058  0.063
```

The operation was about 100x faster on parquet files. Why?

- The parquet files was partitioned by `CheckoutYear`. This means that arrow needs to read only one of the 18 parquet files.
- The binary format can be directly read into memory.
- Since parquet files are column-oriented, arrow only needs to load the columns involved in the calculations (`CheckoutYear`, `MaterialType`, `CheckoutMonth`, and `Checkouts`).

Using duckdb with arrow

This is very straightforward – we just need to turn an arrow dataset into a DuckDB database. This can be achieved by calling `arrow::to_duckdb()`.

```
seattle_pq |>
  to_duckdb() |>
  filter(CheckoutYear > 2018, MaterialType = "BOOK") |>
  group_by(CheckoutYear) |>
  summarize(TotalCheckouts = sum(Checkouts)) |>
```

```

arrange(desc(CheckoutYear)) |>
collect()
#> Warning: Missing values are always removed in SQL aggregation functions.
#> Use `na.rm = TRUE` to silence this warning
#> This warning is displayed once every 8 hours.
#> # A tibble: 5 × 2
#>   CheckoutYear TotalCheckouts
#>   <int>          <dbl>
#> 1      2022      2431502
#> 2      2021      2266438
#> 3      2020      1241999
#> 4      2019      3931688
#> 5      2018      3987569

```

The “conversion” to duckDB doesn’t involve any memory copying, making the operation seamless.